2026-05-05-p1-f08-plan-mode-design

Phase 1 / Feature 8 — Plan Mode

Date: 2026-05-05 **Status:** Approved (brainstorming, auto-approved per programme cadence)
Programme: CLI-Agent Fusion — Phase 1 port from claude-code

---

1. Goal

Add claude-code-style Plan Mode to HelixCode: a read-only operational mode where destructive tools are blocked unless the agent's plan has been approved by the user. Both agent (`EnterPlanMode` tool) and user (`/plan` slash command) can switch into plan mode; `ExitPlanMode` agent tool and `/plan reject` (or `/plan approve`+execute) return to normal execution. Hooks fire on plan approval/reject so users can wire policy enforcement (per F05). Allow-list of always-safe tools (`Read`, `Glob`, `Grep`, `View`, `LSPGetDiagnostics`, `TaskOutput`, `TaskStop`, `WebFetch`, `WebSearch`); everything else blocked unless explicitly approved as a plan action.

This ports claude-code's Plan Mode pattern, building on the existing heavyweight `internal/workflow/planmode/` package (Mode/ModeController/Plan/Planner already exist) by adding the missing tool-gating layer.

2. Architecture

Extend `internal/workflow/planmode/` with a new `gating.go` file holding `ToolGate` (knows the allow-list + per-tool key-arg map, queries `ModeController` + `Planner`, returns blocked/allowed for any tool call). Add `EnterPlanModeTool` + `ExitPlanModeTool` agent tools that wrap `ModeController.TransitionTo(...)`. Add `/plan` slash command exposing `show`/`approve [<id>]`/`reject`/`status` to interactive users. Wire `ToolRegistry.Execute` with `SetPlanModeGate(*ToolGate)` setter (mirrors F07's `SetBackgroundManager`); the registry consults the gate before any tool execution and returns `ErrPlanModeGated` if blocked. Hook integration via the existing F05 hook system (new `OnPlanApproval`, `OnPlanReject` hook types).

Boundary discipline: - `ToolGate` is the only place that knows the allow-list, key-arg map, and the gating decision. ToolRegistry asks one question; gate answers. - `Planner`/`Plan` machinery from existing `planmode/` is reused as-is — no refactor of the heavyweight option-presenter code. - `EnterPlanMode`/`ExitPlanMode` agent tools are pure mode-transition wrappers — they don't know about the gate or planner internals. - `/plan` slash + agent tools are sibling consumers of `Planner` + `ModeController`. Neither calls the other.

3. Components

3.1 New files

File	Responsibility
`helix_code/internal/workflow/planmode/gating.go`	`ToolGate` struct + allow-list + key-arg map + `IsBlocked()`, `MatchApprovedAction()`
`helix_code/internal/workflow/planmode/gating_test.go`	Unit tests for gate logic
`helix_code/internal/tools/plan_tools.go`	`EnterPlanModeTool`, `ExitPlanModeTool` (both implement `Tool`)
`helix_code/internal/tools/plan_tools_test.go`	Unit tests for the agent tools
`helix_code/internal/tools/types_planmode.go`	`ErrPlanModeGated` sentinel
`helix_code/internal/commands/plan_command.go`	`/plan` slash command
`helix_code/internal/commands/plan_command_test.go`	Unit tests for the slash command
`helix_code/internal/commands/builtin/plan_register_test.go`	Test for `RegisterBuiltinCommandsWithPlanMode`
`helix_code/tests/integration/planmode_gating_test.go`	`//go:build integration` end-to-end test
`challenges/p1-f08-plan-mode/CHALLENGE.md` + `run.sh`	Runtime-evidence Challenge

3.2 Modified files

File	Change
`helix_code/internal/tools/registry.go`	`SetPlanModeGate(*planmode.ToolGate)` method; `Execute` consults gate; returns `ErrPlanModeGated` on block. Foreground + background paths both gated.
`helix_code/internal/commands/builtin/register.go`	New `RegisterBuiltinCommandsWithPlanMode(registry, planner, mc)` mirroring the Hooks/MCP/Tasks pattern. Add `"plan"` to `GetBuiltinCommandNames`.

File	Change
helix_code/internal/hooks/hook.go	Add OnPlanApproval, OnPlanReject hook type constants.
helix_code/cmd/cli/main.go	Construct ToolGate, wire to registry + slash registration.

3.3 ToolGate (internal/workflow/planmode/gating.go)

```
package planmode

// AllowList is the default set of always-safe tool names — read-
        only/inspection tools.
var DefaultAllowList = []string{
    "Read", "Glob", "Grep", "View", "LSPGetDiagnostics",
    "TaskOutput", "TaskStop", "WebFetch", "WebSearch",
}

// DefaultKeyArgMap maps each destructive tool name to the param
        key that
// identifies its target. An approved plan action matches an
        Execute call only
// when the values for that key are equal (where defined).
var DefaultKeyArgMap = map[string]string{
    "Edit":         "file_path",
    "Write":        "file_path",
    "MultiEdit":    "file_path",
    "NotebookEdit": "notebook_path",
    "Bash":         "command",
    "shell":        "command",
}

type ToolGate struct {
    mc        ModeController
    planner   Planner
    allowList map[string]bool
    keyArgs   map[string]string  // toolName → param key for
        matching
}

func NewToolGate(mc ModeController, planner Planner) *ToolGate
func (g *ToolGate) IsBlocked(toolName string, params
        map[string]any) (blocked bool, reason string)
func (g *ToolGate) MatchApprovedAction(toolName string, params
        map[string]any) (*PlanAction, bool)
func (g *ToolGate) WithAllowList(extra []string) *ToolGate    //
        for YAML override (deferred)
```

Decision logic for `IsBlocked`: 1. If `mc.GetMode() != ModePlan`, return `(false, "")`. 2. If `toolName ∈ allowList`, return `(false, "")`. 3. Get the active plan from `planner.ActivePlan()` (or equivalent). If nil, return `(true, "no active plan")`. 4. Walk plan actions: find one with `ToolName == toolName`, `Approved == true`, `Executed == false`. If key-arg defined for this tool, also require matching arg value. 5. If found, return `(false, "")` — the action will be marked Executed by the caller (registry) after a successful Execute. 6. Otherwise return `(true, "no approved plan action authorises this tool")`.

3.4 `Planner` extensions

The existing `planmode.Planner` interface already covers plan creation. F08 needs three method signatures (add to the interface, implement in the existing `DefaultPlanner`): - `ApprovePlan(planID string) error` — sets `Status = PlanApproved`, marks all actions approved, fires `OnPlanApproval`. - `ApproveAction(planID, actionID string) error` — marks one action approved. - `RejectPlan(planID string) error` — sets `Status = PlanRejected`, fires `OnPlanReject`, transitions mode back to Normal.

If `ActivePlan() *Plan` doesn't exist, add it (returns the plan whose status is Pending/Approved).

3.5 Agent tools

```
// EnterPlanModeTool — agent calls this to switch into plan mode.
// Side-effect only: ModeController.TransitionTo(ModePlan).
type EnterPlanModeTool struct{ mc planmode.ModeController }

// ExitPlanModeTool — agent calls this to leave plan mode.
// Optional param: allowed_prompts []string (claude-code parity,
        used by F05 hook policies).
type ExitPlanModeTool struct{ mc planmode.ModeController }
```

Both implement the 6-method `Tool` interface (Name/Description/Schema/Category/Validate/Execute) per the convention established in F07.

3.6 `/plan` slash command

```
/plan                  # alias for /plan show
/plan show             # render active plan: title, description, actions t
/plan approve          # approve all actions in the active plan
/plan approve <id>     # approve one action by ID
/plan reject           # reject the active plan, exit plan mode
/plan status           # current mode + active plan summary
```

Mirrors the F02–F07 slash-command-with-subcommands pattern. Uses the `*CommandContext` / `*CommandResult` interface adopted in F06–F07.

3.7 Registry wiring

```
func (r *ToolRegistry) SetPlanModeGate(g *planmode.ToolGate)
        { ... }
```

In `Execute` (after the F07 `run_in_background` shortcut, before fireBefore):

```
if r.planModeGate != nil {
    blocked, reason := r.planModeGate.IsBlocked(name, params)
    if blocked {
        return nil, fmt.Errorf("%w: %s (%s)", ErrPlanModeGated,
        name, reason)
    }
}
```

The same gate consultation happens inside `executeInBackground` so background dispatch is also gated.

3.8 Hook types

In `internal/hooks/hook.go`, add to the existing `HookType` const block:

```
OnPlanApproval HookType = "OnPlanApproval"
OnPlanReject   HookType = "OnPlanReject"
```

The `Planner.ApprovePlan` and `RejectPlan` implementations fire these via the existing `hooks.Manager` (passed in via constructor).

4. Data flow

4.1 Agent enters plan mode

```
agent calls EnterPlanMode {}
  └─ EnterPlanModeTool.Execute
       └─ mc.TransitionTo(ModePlan)
            └─ ModeController callbacks fire (announce mode change)
       └─ return {mode: "plan", message: "Plan mode active. Propose a plan
```

4.2 Agent proposes plan

The agent calls existing `Planner` APIs (already in `planmode/planner.go`) — F08 doesn't add a new agent surface for plan creation. The agent constructs a `Plan` with `[]PlanAction` and calls `Planner.SubmitPlan(plan)` (existing API or equivalent).

4.3 User approves

```
user types "/plan approve"
  └─ PlanCommand.Execute({Args: ["approve"]})
       ├─ planID := planner.ActivePlan().ID
       ├─ planner.ApprovePlan(planID)
       │    ├─ Status = PlanApproved; all actions approved
       │    └─ hooks.Manager.Trigger(OnPlanApproval, plan)
       └─ return "Plan approved (N actions)."
```

User can also `/plan approve <action-id>` to approve a single action. The plan's overall status stays `Pending` until all actions are approved or `/plan approve` (no arg) is run.

4.4 Agent executes destructive tool against approved plan

```
agent calls "Edit" {file_path: "foo.go", ...}
  └─ ToolRegistry.Execute(ctx, "Edit", params)
       ├─ planModeGate.IsBlocked("Edit", params)
       │    ├─ mode == ModePlan
       │    ├─ "Edit" ∉ allowList
       │    ├─ active plan; walk actions → find one with ToolName=="Edit"
       │    └─ return (false, "")  // matched approved action
       ├─ existing fireBefore + Execute path runs
       ├─ on success: planModeGate.MarkExecuted(actionID) — the matched ac
       └─ return result
```

4.5 Agent tries unapproved destructive tool

```
agent calls "Bash" {command: "rm -rf /tmp/x"} (no matching plan action)
  └─ ToolRegistry.Execute
       ├─ planModeGate.IsBlocked → (true, "no approved plan action authori
       └─ return ErrPlanModeGated wrapped with name + reason
```

The agent sees the error, can either: 1. Propose a new plan action and ask user to approve it. 2. Call `ExitPlanMode` to return to normal mode (if user has authorised).

4.6 User rejects

```
user types "/plan reject"
  └─ PlanCommand.Execute({Args: ["reject"]})
       ├─ planner.RejectPlan(activePlanID)
       │    ├─ Status = PlanRejected
       │    └─ hooks.Manager.Trigger(OnPlanReject, plan)
       ├─ mc.TransitionTo(ModeNormal)
       └─ return "Plan rejected. Returning to normal mode."
```

4.7 Agent exits plan mode

```
agent calls ExitPlanMode {allowed_prompts: ["..."]}
  └─ ExitPlanModeTool.Execute
       ├─ if claude-code semantics: store allowed_prompts on planner state
       └─ mc.TransitionTo(ModeNormal)  // plan stays in archive; new plans
```

5. Error handling, edge cases, anti-bluff

5.1 Error sentinel

```go
var ErrPlanModeGated = errors.New("tools: blocked by plan mode")
```

Wrapped with the tool name and reason for diagnostic clarity:

```go
return nil, fmt.Errorf("%w: %s (%s)", ErrPlanModeGated, name,
        reason)
```

5.2 Concurrency invariants

- `ModeController` already has `sync.RWMutex` (existing).
- `ToolGate` doesn't store mutable state of its own — it queries `mc` and `planner` per call. No new lock needed.
- `Planner.ApprovePlan/RejectPlan` mutations under the planner's existing `sync.RWMutex`.
- Hook firing happens after the lock release to avoid holding locks across user-supplied hook code.

5.3 Edge cases

- **Empty plan approved** (zero actions): `ApprovePlan` succeeds, status becomes Approved, no actions to authorise. Any destructive tool call hits `IsBlocked → (true, ...)`. The user must add actions or reject.
- **All actions executed**: subsequent destructive tool calls are blocked (no unexecuted approved action remains). The agent should propose a new plan or `ExitPlanMode`.
- **Mode change mid-execution**: rare (gate checks happen synchronously in Execute). If `mc` state changes between check and execute, the result is inconsistent but bounded — at most one extra tool runs in the wrong mode.
- **Background tasks in plan mode**: `executeInBackground` consults the same gate. If the destructive tool would be blocked in foreground, it's also blocked when invoked with `run_in_background:true`.
- **Allow-list tool with side effects**: `Read` is allow-listed but reads can leak path information. Not a concern at the gating layer — the allow-list is what claude-code defines; users can shrink it via the WithAllowList override.

5.4 Resource limits

No new resources. Gate is stateless beyond the `mc`/`planner` references. Allow-list and key-arg map are O(K) lookup where K is small.

5.5 Anti-bluff (CONST-035, Article XI §11.9)

- Challenge in T13: spawn a real `Bash` background task in plan mode WITHOUT an approved plan, assert blocked. Then approve a `Bash` action with matching `command`, assert allowed, assert tool actually runs.
- Tool name + key-arg matching enforced by tests with multiple actions and verification of which matched.
- No metadata-only PASS: every test asserts on `IsBlocked` returning the expected `(blocked, reason)` pair AND on a real subsequent Execute call producing the expected outcome.
- Anti-bluff smoke `grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/workflow/planmode/gating.go internal/tools/plan_tools.go internal/tools/types_planmode.go internal/commands/plan_command.go` must return empty.

5.6 Logging

- Mode transitions log via existing `internal/logging` (already done by `ModeController`).
- Gate decisions log at DEBUG (with tool name + reason).
- Hook firing on Approve/Reject logs at INFO (handled by `hooks.Manager`).

6. Testing

6.1 Unit tests (mocks allowed)

`gating_test.go`: - TestToolGate_NormalModeNeverBlocks - TestToolGate_AllowListPasses - TestToolGate_NoActivePlanBlocks - TestToolGate_ApprovedActionAllows - TestToolGate_KeyArgMismatchBlocks - TestToolGate_ExecutedActionDoesNotReauthorise - TestToolGate_RejectedPlanBlocksAll - TestToolGate_WithAllowListExtends

`plan_tools_test.go`: - TestEnterPlanModeTool_TransitionsToPlanMode - TestExitPlanModeTool_TransitionsToNormal - TestEnterPlanModeTool_FromInvalidStateErrors

`plan_command_test.go`: - TestSlashPlan_ShowEmptyPlan - TestSlashPlan_ShowActivePlan - TestSlashPlan_ApproveAll - TestSlashPlan_ApproveSingleAction - TestSlashPlan_RejectExitsPlanMode - TestSlashPlan_StatusReportsMode - TestSlashPlan_UnknownSubcommandErrors

Registry tests (extend existing test file): - TestRegistry_PlanModeGateBlocksDestructive - TestRegistry_PlanModeGateAllowsAllowList - TestRegistry_PlanModeGateAllowsApprovedAction - TestRegistry_PlanModeGateBlocksOnBackgroundDispatch (T07 path)

`plan_register_test.go`: - TestRegisterBuiltinCommandsWithPlanMode

6.2 Integration test (`-tags=integration`)

`tests/integration/planmode_gating_test.go`: - `TestPlanMode_BlocksUnapprovedBash` — real registry, real ModeController; transition to plan mode; assert `Bash` blocked. Approve a plan action with matching command. Assert subsequent `Bash` allowed and runs (asserts on actual subprocess output). - `TestPlanMode_AllowListPasses` — transition to plan mode; call `Glob`; assert it runs without approval. - `TestPlanMode_ExitReturnsToNormal` — agent calls `ExitPlanMode`; subsequent `Bash` runs without approval.

6.3 Challenge

`challenges/p1-f08-plan-mode/`: 1. Build the harness. 2. Run a small program that: a. Constructs registry + ModeController + Planner + ToolGate. b. Transitions to ModePlan. c. Tries to execute `Bash echo hi` → expects `ErrPlanModeGated`. d. Submits a plan with action `Bash command=echo hi`, approves it. e. Tries to execute `Bash echo hi` → expects success, captures "hi" output. f. Tries to execute `Bash echo bye` (different command) → expects `ErrPlanModeGated` (key-arg mismatch). g. Calls `ExitPlanMode`, then `Bash echo bye` → expects success. 3. Anti-bluff smoke clean. 4. Cross-compile linux clean.

Pasted runtime evidence to `docs/improvements/06_phase_1_evidence.md`.

7. Cross-platform

No new platform-specific code. `ModeController`/`Planner`/`ToolGate` are pure Go. Cross-compile check: linux native + windows compile (the existing pre-existing CGO failures in `internal/tools/git`, etc., remain documented out-of-scope).

8. Out of scope (deferred)

- YAML override of allow-list / key-arg map. The `WithAllowList(extra)` builder hook is in place but no YAML loader yet — defer to F08.5 if needed.
- Plan persistence across sessions (the existing `StateManager` is in-memory; durable plans need DB integration).
- `EnterPlanMode` `allowed_prompts` parameter — accepted by the tool but currently ignored. Future feature: F05 hook can consult the stored allowed_prompts to gate user-prompt-driven exits.
- A standalone `helixcode plan` cobra subcommand — the `/plan` slash is the user surface; a top-level cobra command would only operate on a session and adds no value over `/plan` (same constraint discovered in F07-T08).

9. Constitutional compliance

- **CONST-033** (no power management): N/A — F08 doesn't spawn subprocesses.
- **CONST-035 / Article XI §11.9** (anti-bluff): every PASS in the gate's tests carries runtime evidence; the Challenge spawns a real subprocess gated correctly.
- **CONST-036–040** (LLMsVerifier, providers, capabilities): orthogonal.
- **CONST-042** (no-secret-leak): tool params are not logged at INFO; gate decisions logged at DEBUG only with tool name (no params).
- **CONST-043** (no-force-push): non-force pushes to all four remotes per programme convention.

10. Open questions resolved during brainstorming

Q	Answer
Q1: where does gating live	(A) `internal/workflow/planmode/gating.go`
Q2: feature scope	(C) Full parity: gating + Enter/ExitPlanMode tools + /plan slash + hooks
Q3: allow-list vs deny-list	(A) Allow-list (default-deny, claude-code's design)
Q4: who can enter plan mode	(B) Both `EnterPlanMode` agent tool + `/plan` slash command
Q5: action matching strictness	(B) Tool name + per-tool key-arg match, fall back to name-only when no key-arg defined